

Git Troubleshooting Guide — "My .git broke after reopening VS Code"

What actually happened (root cause)

You closed VS Code with the project open, started a different project, then reopened this project 6 hours later. Somewhere in that process, the `.git` folder in your working directory either got wiped, corrupted, or a nested duplicate project folder was created (e.g. `harit-backend\harit\`) — a full second copy of the project sitting inside the first one, complete with its own `.git`, `.venv`, `.env`, and `db.sqlite3`).

Symptom: `git status` / `git add .` says `fatal: not a git repository`, even though a `.git` folder appears to exist in the VS Code file explorer.

Step 1 — Confirm where you are and whether `.git` really exists

```
powershell  
  
pwd  
Test-Path .git
```

- `False` → `.git` is genuinely missing/broken → go to Step 2
 - `True` but git still errors → folder may be corrupted → skip to Step 4
-

Step 2 — Check for a nested duplicate folder

This was the actual root cause last time — a project-inside-a-project.

```
powershell  
  
dir -Force
```

Look for another folder with the same project files inside it (e.g. `harit-backend\harit\`). If found, check it:

```
powershell  
  
cd <nested-folder>  
Test-Path .git
```

If **that one** has a working `.git`, that's your real project — work from there instead of re-initializing from scratch.

Step 3 — If truly no `.git` anywhere, reconnect to GitHub (don't commit yet)

powershell

```
git init
git remote add origin https://github.com/GtTanishSharma/RenSetu
git fetch origin
```

Step 4 — Compare local vs remote BEFORE touching anything

powershell

```
git diff main origin/main --stat
git log origin/main --oneline -10
```

- Diff shows mostly **additions** (remote has files you don't) → remote is the source of truth
- Diff shows **your local files disappearing** → you might have newer local work, don't reset blindly

Step 5 — Sync local to match remote safely

Only do this once you've confirmed remote is the source of truth (usually true, since it's live on Render):

powershell

```
git reset --hard origin/main
```

Step 6 — Recover real secrets/data if left in a nested folder

powershell

```
copy <nested-folder>\.env . -Force
copy <nested-folder>\db.sqlite3 . -Force
```

Verify before deleting anything:

powershell

```
type .env
```

Step 7 — Delete the leftover duplicate folder (only after confirming copies worked)

powershell

```
Remove-Item -Recurse -Force <nested-folder>
```

Step 8 — Rebuild your virtual environment

The `.venv` often lives inside the folder that just got deleted, so it needs to be rebuilt:

powershell

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

If PowerShell blocks the activate script:

powershell

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Step 9 — Test locally

powershell

```
python manage.py runserver
```

Open `http://127.0.0.1:8000/` and confirm real data shows up (dealers, guides, etc.).

Step 10 — Push safely

powershell

```
git add .
git commit -m "your message"
git push
```

If it says "no upstream branch":

powershell

```
git push --set-upstream origin main
```

How to prevent this from happening again

1. **Always open the project via `File > Open Folder` in VS Code pointing to the exact same path** — don't let VS Code default to "recent" or create a new folder structure.
2. **Before closing VS Code, always run `git status`**. If it's not clean, commit or stash. An abrupt close mid-git-operation is likely what corrupted `.git` last time.
3. **Never nest one git project inside another**. If you see a folder containing another copy of itself, stop and investigate before doing anything.
4. **Push often** — even small WIP commits. If local `.git` breaks again, you lose almost nothing since GitHub already has it.
5. **Check your Render dashboard** any time local stuff gets messy, to confirm production is still healthy — force pushes/resets can't hurt you if you check the diff first (Step 4).